Group 57
Vashti Pillay - u25041887
Rayhaan Ahmed -
Njabulo Nhlengethwa - u24981712

Task 1: Design the System

TaskForge is a software that models emergency responders management. At the root is an incident command that oversees divisions, (like Fire or Hazmat). Divisions are then broken down into squads and those squads are made up of individual units, e.g. a fire Engine or Medical Team.

- The software needs to solve four problems. A dispatcher must be able to:
    - issue out an incident command to a single unit or an entire division in the same way (Composite).
    - search or view the incident command structure in different ways (Iterator).
    - track the status of each unit while also preventing impossible commands (State).
    - add temporary, combining rules to any unit during rollout without needing to create new types in the code (Decorator).

Ownership Reasoning

- ResponderGroup owns its children - Composition but transferable
    - A group is responsible for the lifetime of every child in the children's list, its destructor deletes all of them.
    - But the ownership can move to another ResponderComponent when remove() and add() are used, remove() only erases the child and the caller becomes responsible for the lifetime of the child. So at any given moment, a component belongs to exactly one group, never zero or more than one, it is just an explicit remove() and add().
- ResponderDecorator owns wrapped - Composition
    - A decorator does nothing without what it's wrapping, wrapped is set once in the constructor and deleted with no conditions in the destructor.
- ResponderUnit owns currentState - Composition
    - A unit always has exactly one current state object and on every transition, the one state is deleted before the new one is assigned.

Example Hierarchy (Used in Scenario)

Level 1 Group : **Division (Fire Department)**

- Level 2 Group : **Squad (Hazardous Material Team)**
    - Level 3 Leaf : Responder Unit (Fire Boat)
    - Level 3 Leaf : Responder Unit (Unmanned Areal System)
- Level 2 Group : **Squad (Fire Suppression Team)**
    - Level 3 Leaf : Responder Unit (Ladder Fire Truck)
    - Level 3 Leaf : Responder Unit (Helicopter)

Level 1 Group : **Division (Search and Rescue)**

- Level 2 Group : **Squad (Urban SAR)**
    - Level 3 Leaf : Responder Unit (Canine Search)
    - Level 3 Leaf : Responder Unit (Cave SAR)
- Level 2 Group : **Squad (Ground, Wilderness, Mountain SAR)**
    - Level 3 Leaf : Responder Unit (Tracking)
    - Level 3 Leaf : Responder Unit (Avalanche Rescue)

Level 1 Group : **Division (Ambulance and Medical)**

- Level 2 Group : **Squad (Transport Fleet)**
    - Level 3 Leaf : Responder Unit (Basic Life Support Ambulance)
    - Level 3 Leaf : Responder Unit (Advanced Life Support Paramedic)
- Level 2 Group : **Squad (Triage and Assessment Team)**
    - Level 3 Leaf : Responder Unit (Field Doctor)
    - Level 3 Leaf : Responder Unit (Vital Signs Nurse)

Level 1 Group : **Division (Police and Security)**

- Level 2 Group : **Squad (Perimeter Control)**
    - Level 3 Leaf : Responder Unit (Traffic Control Technician)
- Level 2 Group : **Squad (Crowd Management)**
    - Level 3 Leaf : Responder Unit (Public Order Policing)

- Level 3 Leaf : Responder Unit (Tactical Response Team)

So the Hierarchy would be The **Emergency Response -> Division -> Squad -> Unit**

Pattern	Role	Class
Composite	Component	ResponderComponent
Composite	Leaf	ResponderUnit
Composite	Composite	ResponderGroup (& EmergencyResponse, Division, Squad)
Composite	Client	Main
State	Context	ResponderUnit
State	State	UnitState
State	ConcreteState	DispatchedState, EnRouteState, OnSceneState, ContainedState
Decorator	Component	ResponderComponent
Decorator	ConcreteComponent	ResponderUnit
Decorator	Decorator	ResponderDecorator
Decorator	ConcreteDecorator	BiohazardDecorator, PriorityDecorator
Iterator	Aggregate	ResponderComponent (createIterator)
Iterator	Iterator	Iterator
Iterator	ConcreteIterator	DFSIterator, ActiveUnitIterator

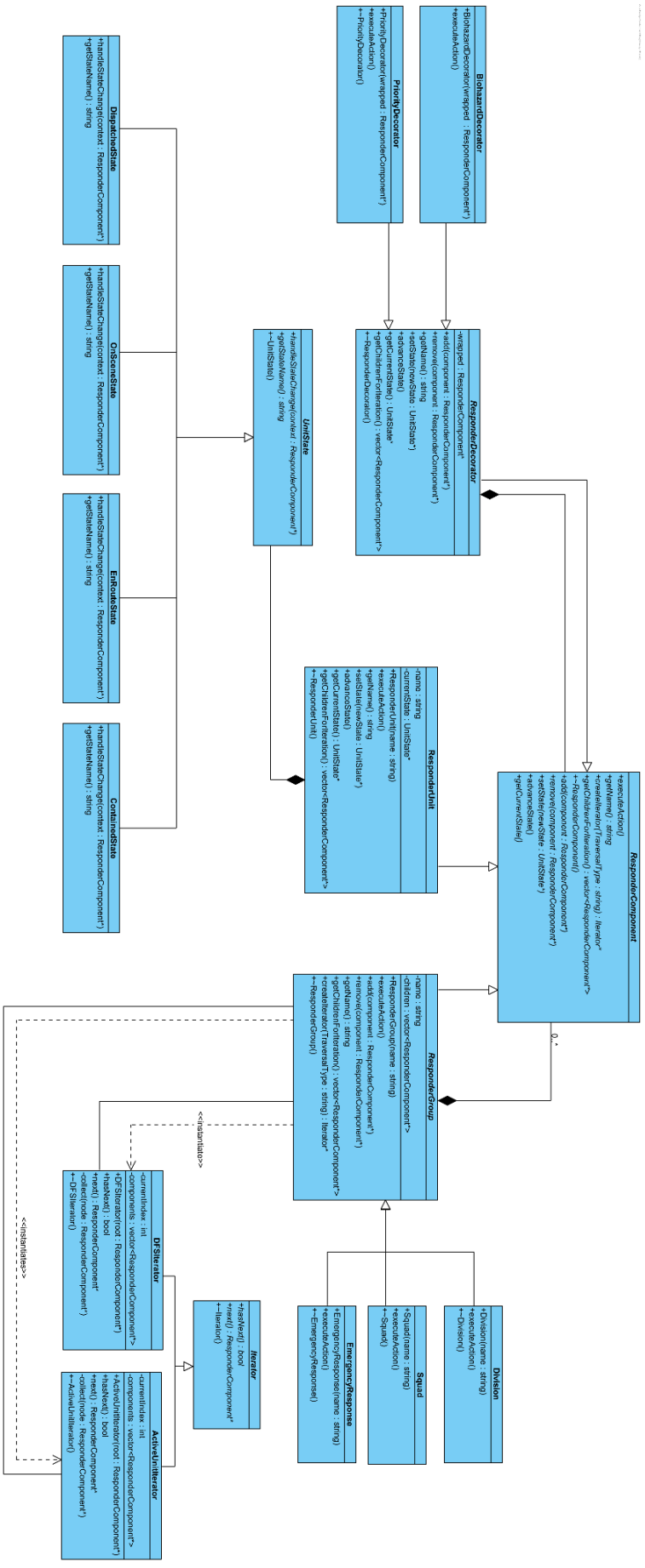
Task 3 - Dynamic Behaviour and Design Decisions

How does existing traversal behave?
Every ConcreteIterator builds its complete list of components once in the constructor. It does this recursively going through the tree of components from the Composite design pattern by getting the children and storing the results in its own vector as it traverses. Then hasNext() and next() are used to go through that list which we have already traversed, making it easier to run the iterator. The client has no idea of how the inner structure of the Components are built but can still traverse it without knowing those inner workings. When a structural

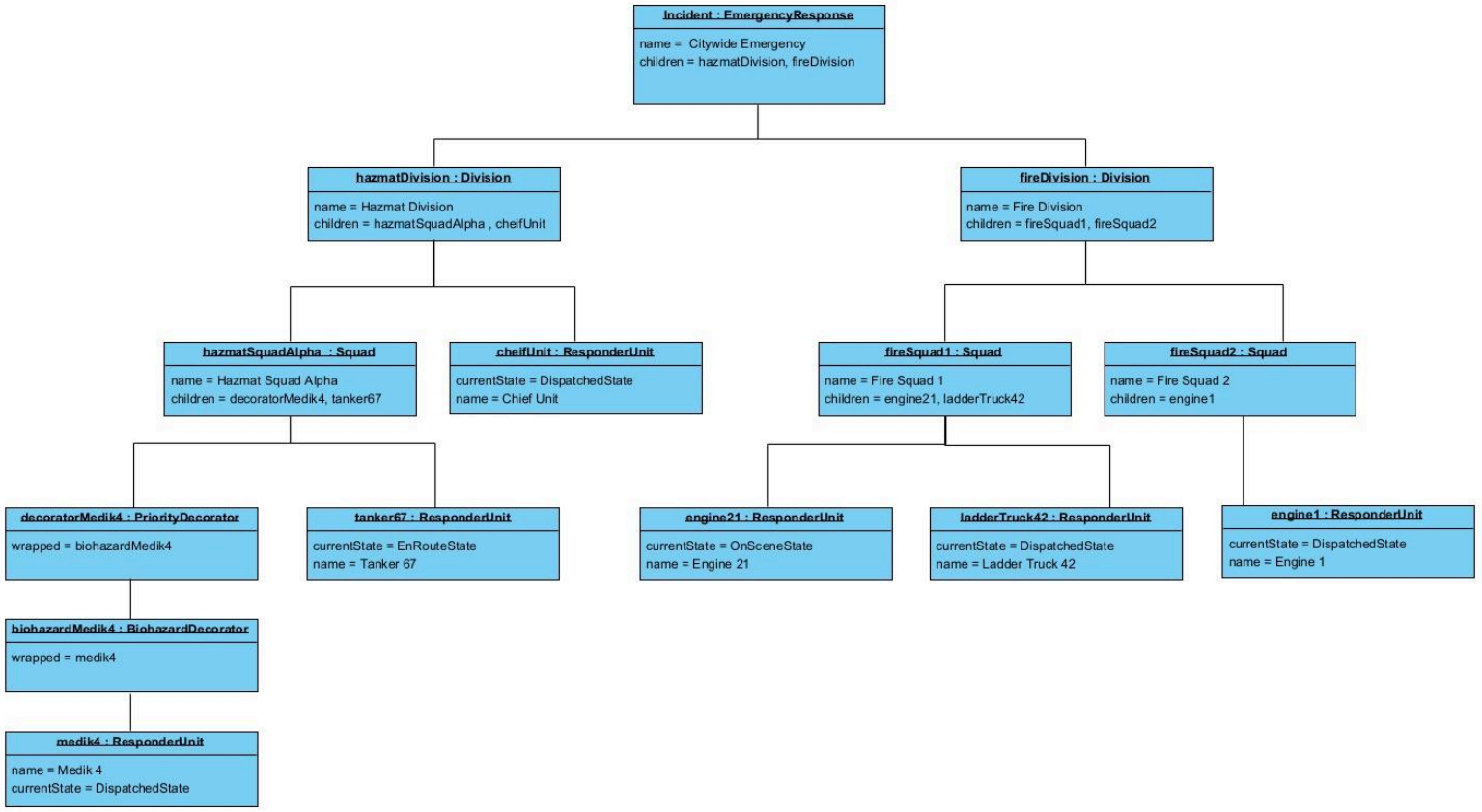
change is made in the client code, call createIterator() after the change is made. In our code: after hazmatDivision->remove(chiefUnit) and fireDivision->add(chiefUnit), we call iterateDFS() again, which constructs a brand-new DFSIterator and rebuilds its list from the current tree. And the chief is shown under the correct new division after that.

Task 4 - UML Diagram Portfolio

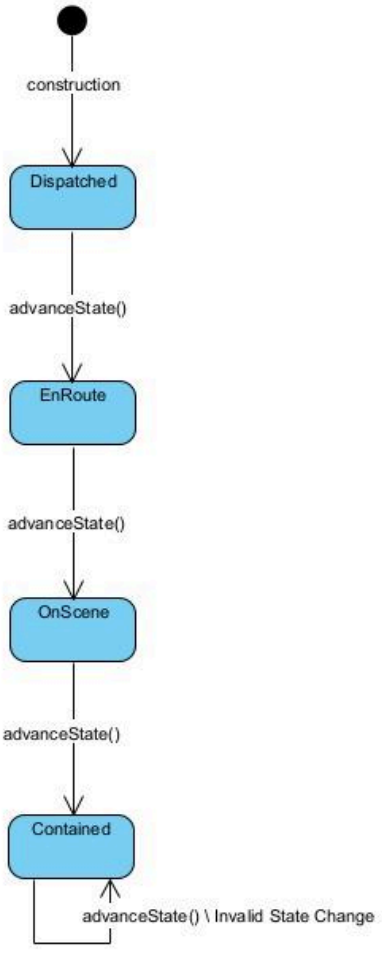
UML DESIGN



Object Diagram

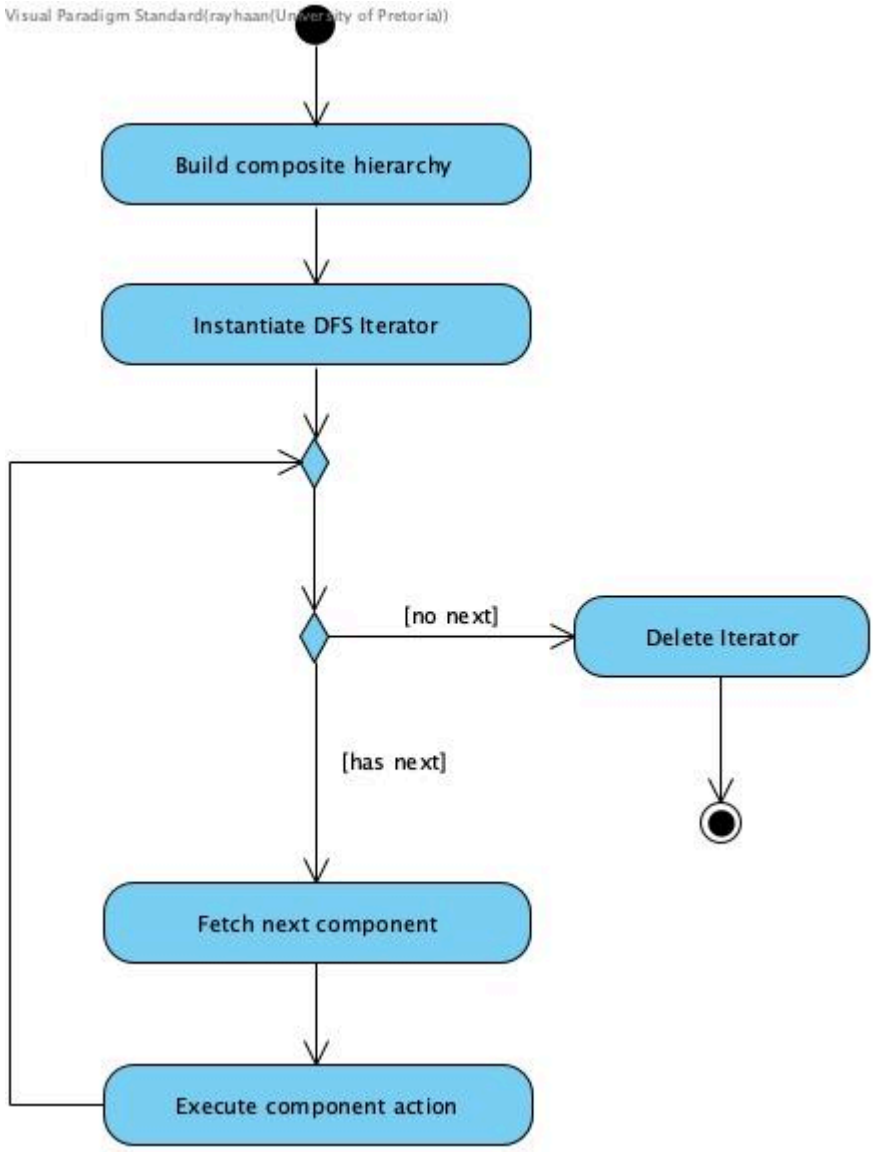


State Diagram

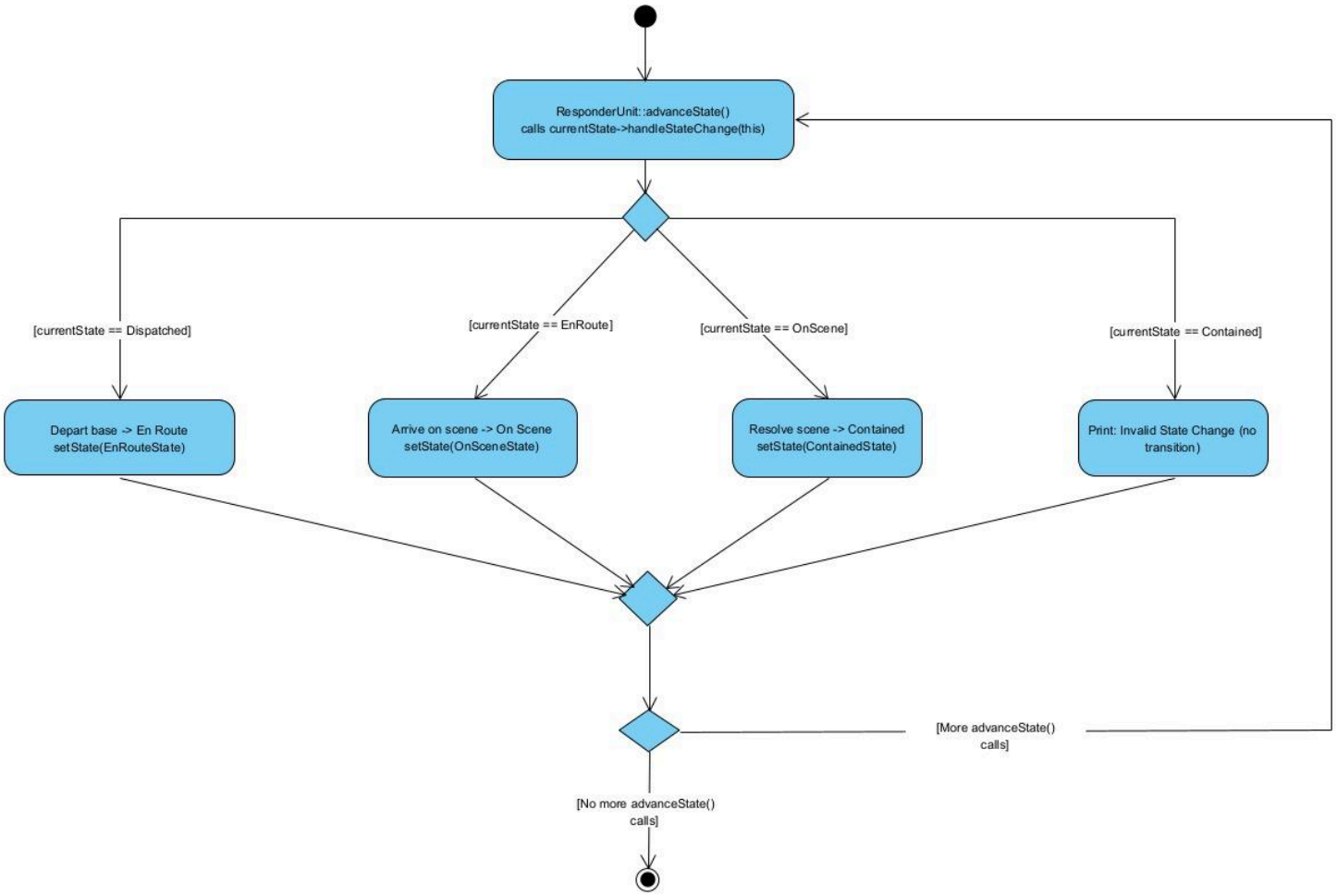


Activity Diagrams

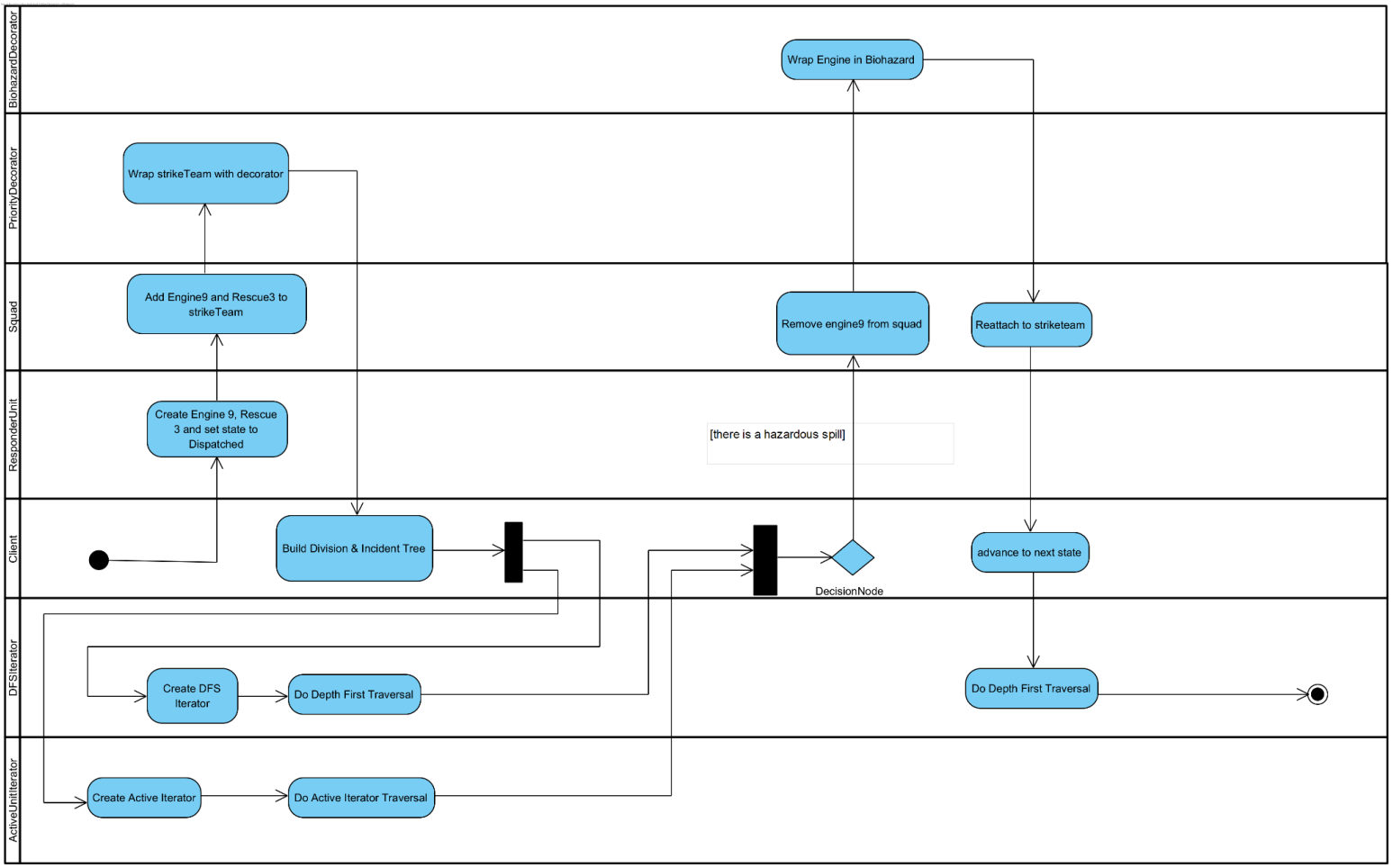
1. Scenario 1 - Showing DFS



2. Scenario 1 - Showing effect of advanceState calls



3. Scenario 2 - Composite / Decorator build and overview of structure



Task 5: Debugging and Memory Investigation

```
--> Entering Strike Team Bravo
[LEAF] Rescue 3 is active. Current state: Dispatched
Biohazard Called. Doing biohazard stuff.
[LEAF] Engine 9 is active. Current state: En Route
==90253==
==90253== HEAP SUMMARY:
==90253==     in use at exit: 0 bytes in 0 blocks
==90253==   total heap usage: 149 allocs, 149 frees, 78,668 bytes allocated
==90253==
==90253== All heap blocks were freed -- no leaks are possible
==90253==
==90253== Use --track-origins=yes to see where uninitialised values come from
==90253== For lists of detected and suppressed errors, rerun with: -s
==90253== ERROR SUMMARY: 47 errors from 16 contexts (suppressed: 0 from 0)
vashti@LAPTOP-BC2LOV30:~/COS214/COS214_Prac4$
```

Fixing a segmentation fault

- Added breakpoint by scenario 2

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./taskforge...
(gdb) break scenario2
Breakpoint 1 at 0x47c7: file main.cpp, line 115.
(gdb) run
Starting program: /home/vashti/COS214/COS214_Prac4/taskforge

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) n
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
----- Hazmat Incident -----
Set Inital States
Changing state to: Dispatched
Changing state to: En Route
Changing state to: Dispatched
Changing state to: On Scene
Changing state to: Dispatched
Changing state to: Dispatched
```

```
Active units after reassignment of chief  (Acitve Unit Traversal)
 Active: Tanker 67
 Active: Medic 4
 Active: Ladder Truck 42
 Active: Engine 1
 Active: Chief Unit
 Active: Fire Division
 Active: Fire Squad 1

Breakpoint 1, scenario2 () at main.cpp:115
115     {
(gdb)
```

Found the segmentation fault - Something is wrong in the ActiveUnitIterator class

```
Program received signal SIGSEGV, Segmentation fault.
0x0000555555559332 in iterateActive (root=0x5555555756c0, label="Sweep B") at main.cpp:167
167             std::cout << " Active: " << iter->next()->getName() << std::endl;
(gdb)
```

```
Program received signal SIGSEGV, Segmentation fault.
0x0000555555559332 in iterateActive (root=0x5555555756c0, label="Sweep B") at main.cpp:167
167             std::cout << " Active: " << iter->next()->getName() << std::endl;
(gdb) bt
#0  0x0000555555559332 in iterateActive (root=0x5555555756c0, label="Sweep B") at main.cpp:167
#1  0x000055555555558c0a in scenario2 () at main.cpp:136
#2  0x00005555555574bb in main () at main.cpp:24
(gdb)
```

What needed fixing

- Forgot to initialise the currentIndex
- hasNext function was indexing past the number of components actually in memory which caused the segmentation fault
    - FIX: Do not use [currentNode + 1] but rather just compare the currentIndex with the size of the component vector so there is no indexing past the end
    -

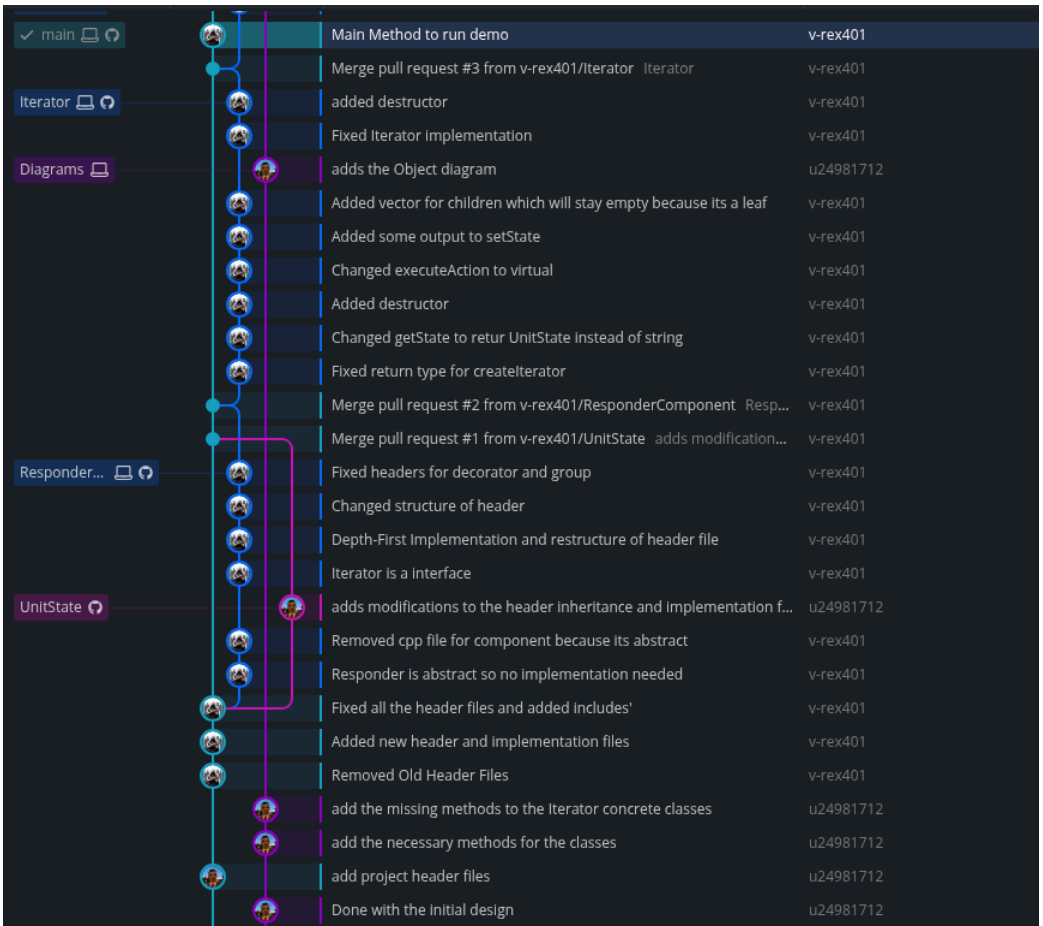
After fixing the segmentation fault

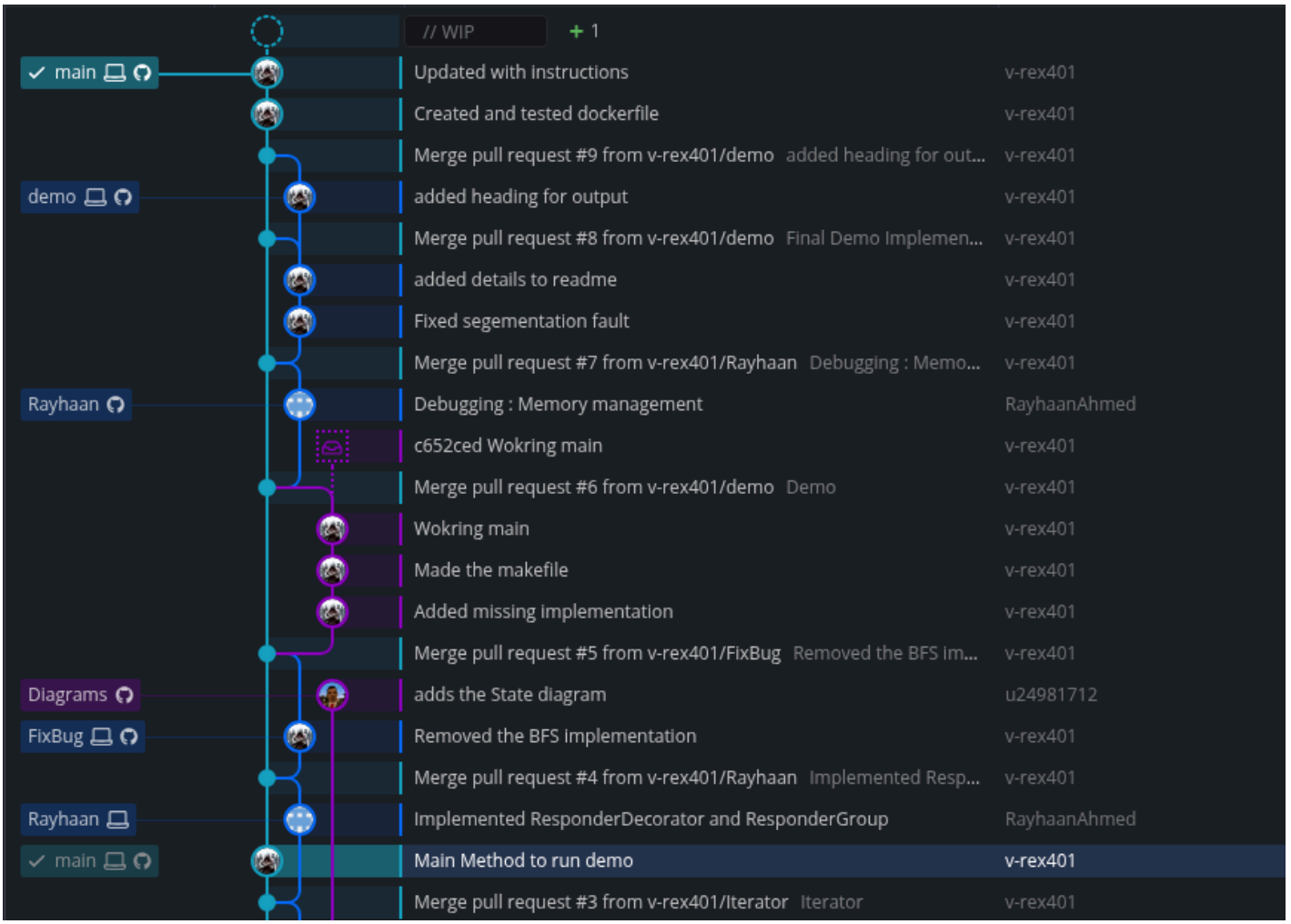
```
--> Entering Escalation Division
Priority Called. Prioritizing stuff.

--> Entering Strike Team Bravo
[LEAF] Rescue 3 is active. Current state: Dispatched
Biohazard Called. Doing biohazard stuff.
[LEAF] Engine 9 is active. Current state: En Route
==96029==
==96029== HEAP SUMMARY:
==96029==     in use at exit: 0 bytes in 0 blocks
==96029==   total heap usage: 149 allocs, 149 frees, 78,668 bytes allocated
==96029==
==96029== All heap blocks were freed -- no leaks are possible
==96029==
==96029== For lists of detected and suppressed errors, rerun with: -s
==96029== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
vashti@LAPTOP-BC2LQV30:~/COS214/COS214_Prac4$
```

Task 6 - Docker and GitHub Workflow

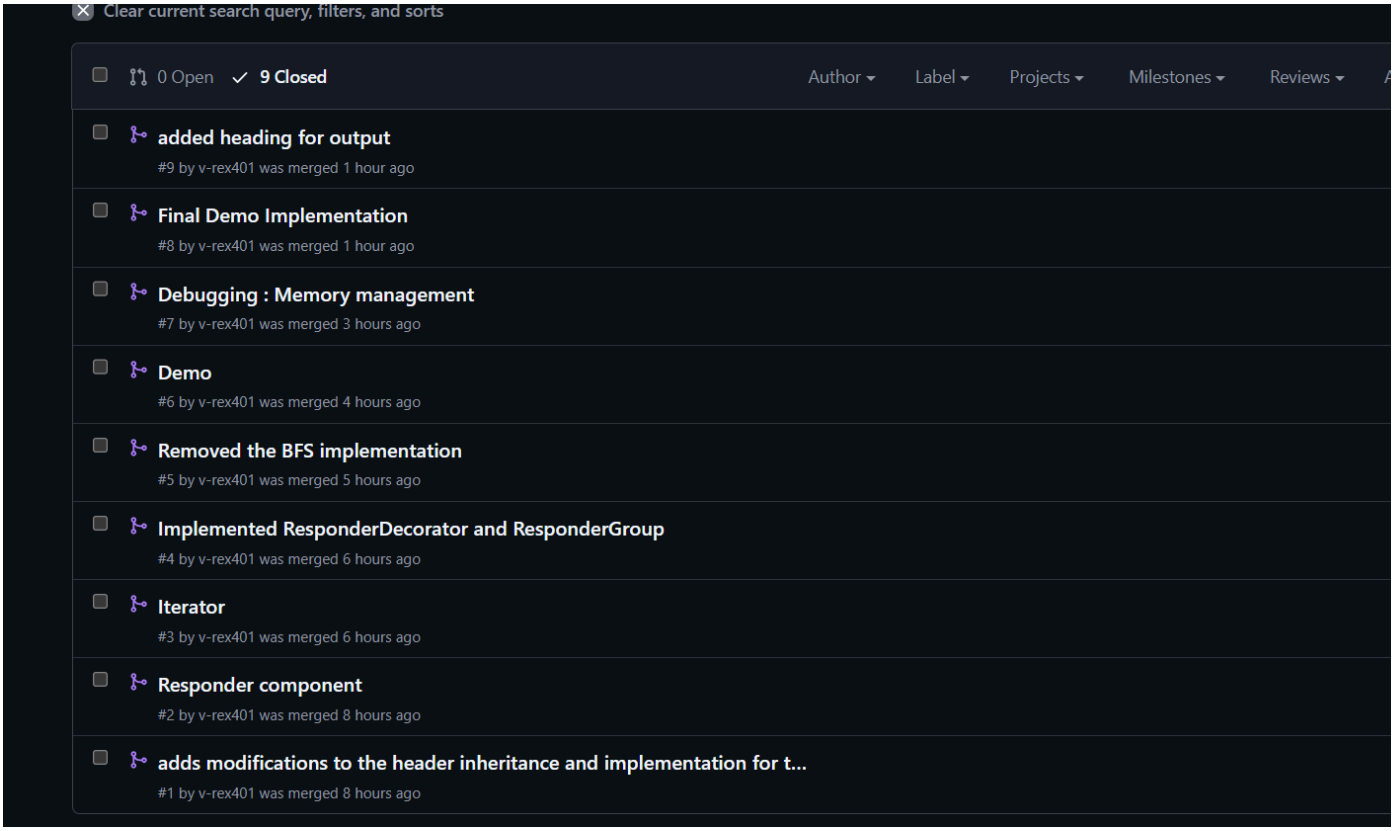
Github Repo: https://github.com/v-rex401/COS214_Prac4.git

Branches





Pull-Requests to merge into main:



Reflection

Delegation of Duties:

Vashti:
- Set up Github repository
- Set up Dockerfile
- Create Makefile
- Create main method and scenarios for demonstration
- Coding:
    - Iterator Design Pattern (DFSIterator & ActiveUnitIterator)
    - ResponderComponent (Abstract class for Composite / Decorator)
    - ResponderUnit (Leaf node in Composite Design Pattern)
- README.md
- Approve pull requests
- Activity Diagram 3

Rayhaan:
- Composite Design Pattern
    - ResponderGroup = Composite
        - Division, Squad and EmergencyResponse
- Decorator Design Pattern
    - ResponderDecorator = Decorator
        - BiohazardDecorator, PriorityDecorator
- Debugging & Memory Management
- Activity Diagram 1

Njabulo:
- Initial UML design
- Object Diagram
- State Diagram
- Code State Design Pattern
    - UnitState
        - Dispatched State, OnSceneState, EnRouteState, ContainedState
- Debugging & Memory Management
- Activity Diagram 2